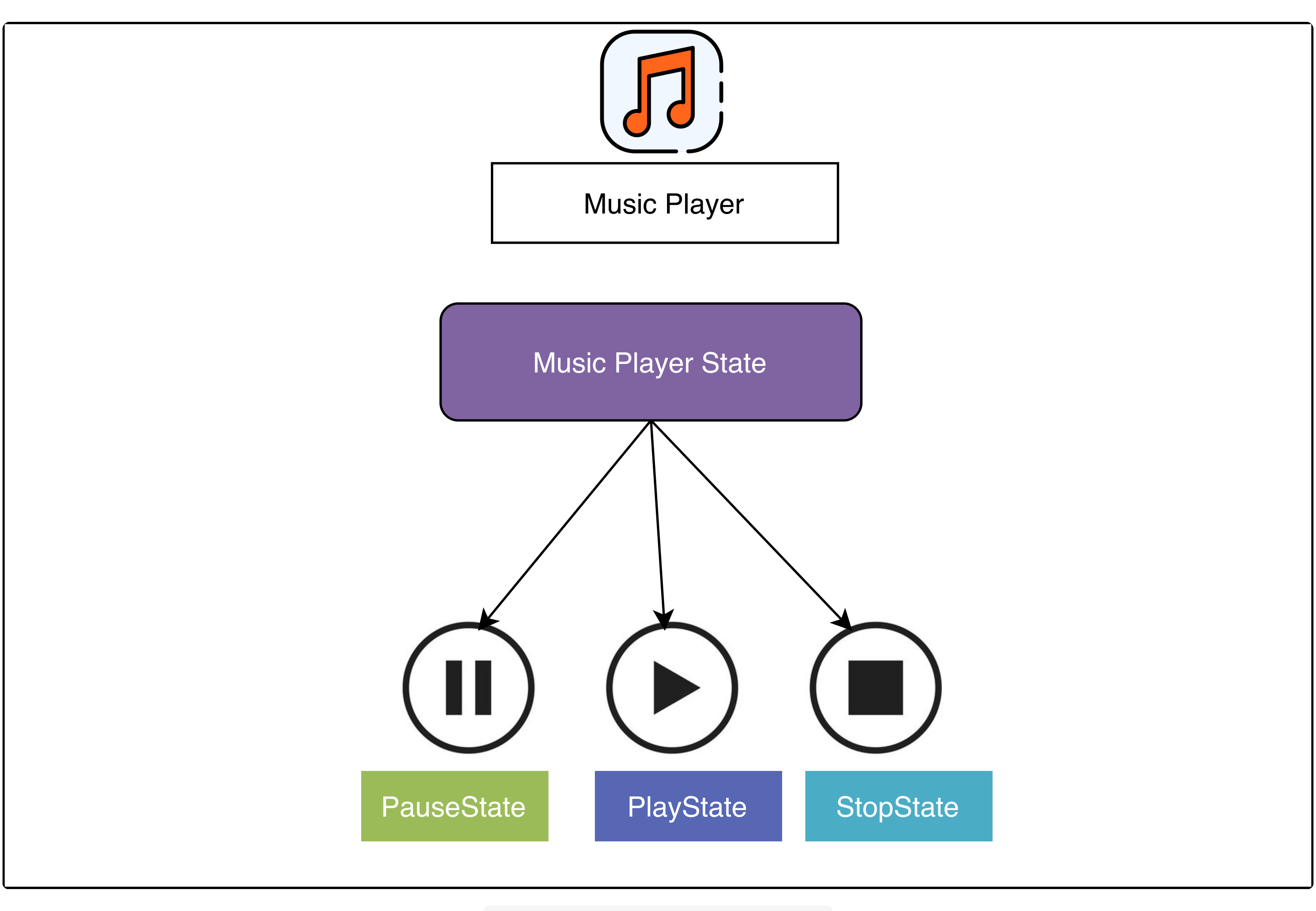
State Pattern

The State design pattern enables an object's behavior to change when its internal state changes. This pattern is used to encapsulate varying behavior for the same routine based on the object's state, allowing an object to change its behavior at runtime without resorting to large conditional statements.

Consider the music player app. The music player can be in any of the following states: stopped, paused, or playing. Depending on the player's current condition, different behaviors are displayed by the buttons labeled "play," "pause," "stop," and "next." It can be difficult and challenging to maintain this implementation when there are a lot of conditional statements, especially when more states or actions are added.

Solution: The State pattern recommends developing distinct classes that implement the same interface or abstract class for each state of the music player, such as `PlayingState`, `PausedState`, and `StoppedState`. The behavior is delegated to the current state object by the MusicPlayer class, which keeps track of it. The Music Player modifies the state object it refers to when the state changes.



State Pattern - Music Player

Real-World Example

Consider a system of traffic lights. There are three possible states for the traffic light: red, green, or yellow. Each state requires a distinct set of actions:

- The traffic light flashes a red signal when it is in the red state.
- It shows green when it is in the Green state.
- It shows yellow when it is in the Yellow state.

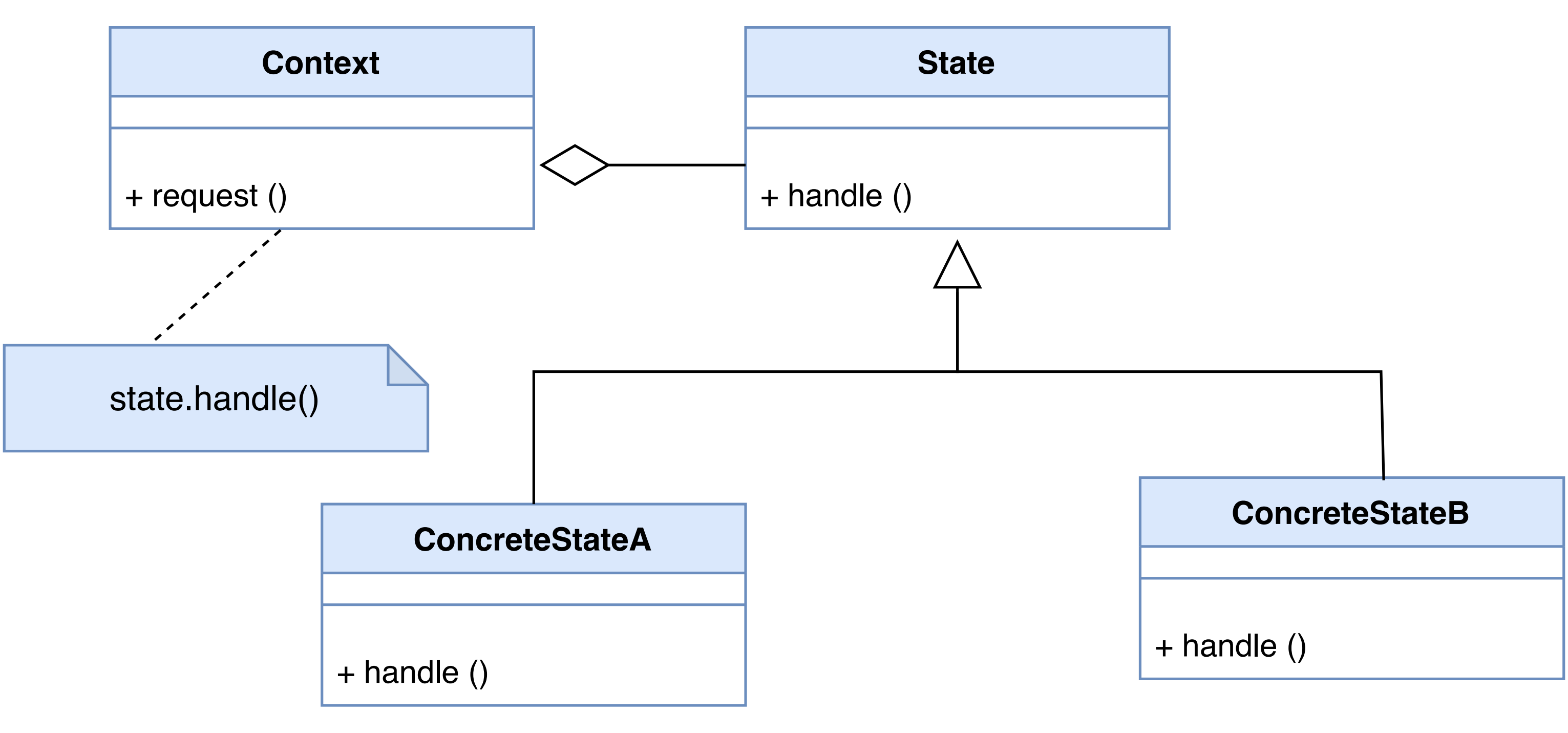
State Pattern - Real Life Example

The traffic light's behavior varies in accordance with the precise sequence in which it changes states—from Red to Green to Yellow to Red.

Structure of State Pattern

The key components of the class diagram include:

- **State Interface**: This interface defines the methods for handling requests, which will be implemented differently by concrete state classes. For example, the State interface, which has `handleRequest()` and other methods.
- **Concrete State Classes**: These classes carry out the implementation of the State interface as well as the methods specified within it. Every class represents a distinct object state. As in a traffic light system, RedState, GreenState, and YellowState.
- **Context Class**: This class has a state and uses the current state object to handle requests specific to that state. It keeps track of a reference to a state object that symbolizes its present condition.
- **Client**: The client interacts with the Context class and has the ability to request a state change.



State Pattern - Class Diagram

The class diagram shows the Context class having a reference to the State interface and the Concrete State classes implementing the State interface. The interaction between the Context and the Concrete States encapsulates the state-dependent behavior.

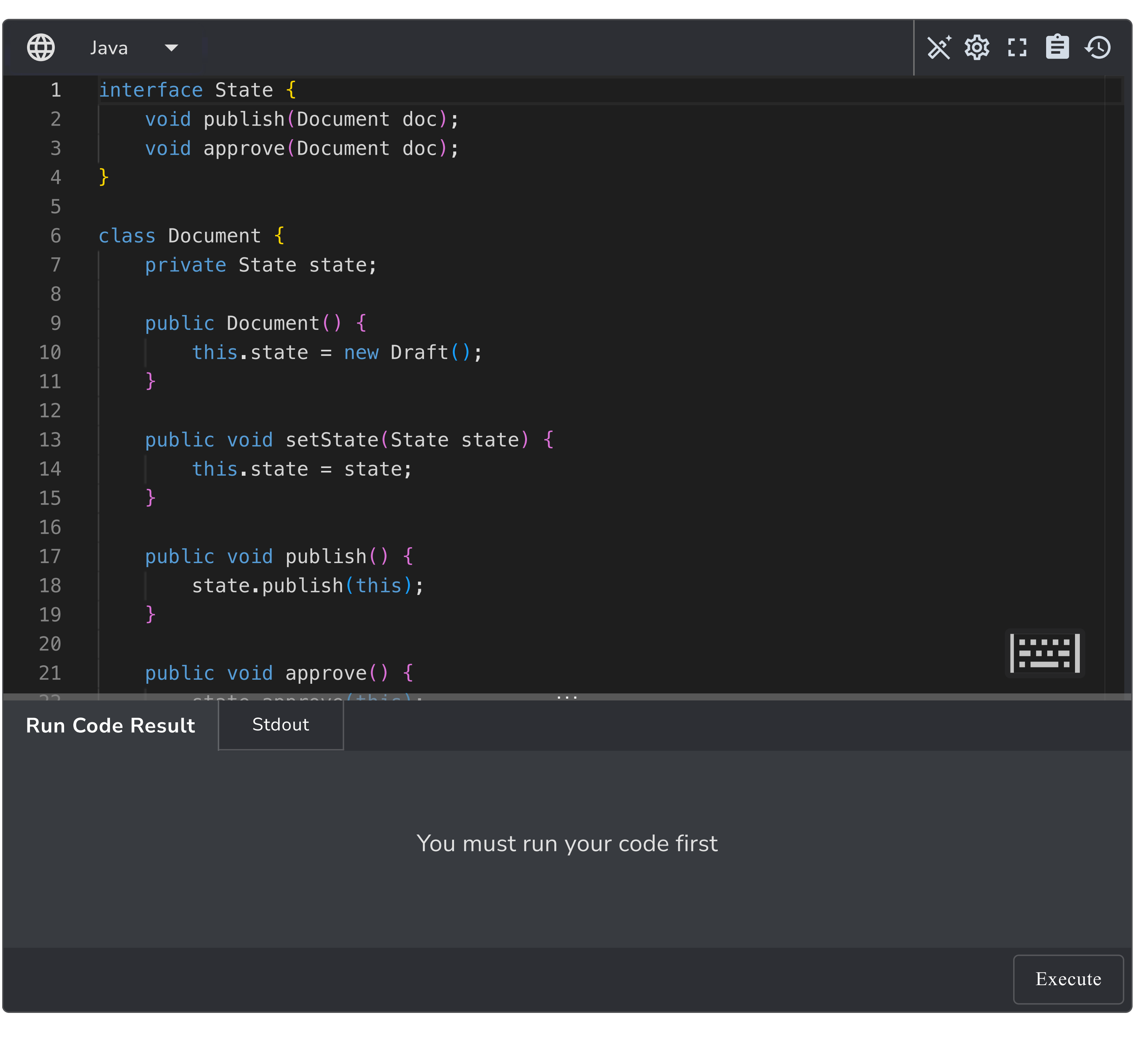
Implementation of State Interface

Let's take the example of a Document class that can be in different states: Draft, Moderation, and Published.

```
// State Interface
interface State {
    void publish(document)
    void approve(document)
}

// Concrete States
class Draft implements State {
    void publish(document) {
        document.state = new Moderation()
    }
    void approve(document) {
        // Draft cannot be approved directly
    }
}

class Moderation implements State {
    void publish(document) {
        // Cannot publish from Moderation without approval
    }
    void approve(document) {
        document.state = new Published()
    }
}

class Published implements State {
    void publish(document) {
        // Already published
    }
    void approve(document) {
        // Already approved
    }
}

// Context Class
class Document {
    State state = new Draft()

    void publish() {
        state.publish(this)
    }

    void approve() {
        state.approve(this)
    }
}

// Usage
document = new Document()
document.publish()  // Changes state to Moderation
document.approve()  // Changes state to Published
```

- **State Interface:** Defines methods for state-specific behaviors (`publish`, `approve`).
- **Concrete States (Draft, Moderation, Published):** Implement behaviors specific to a document's state.
- **Context (Document):** Maintains a reference to the current state and delegates state-specific requests to this state object.

Implementation



```java
interface State {
    void publish(Document doc);
    void approve(Document doc);
}

class Document {
    private State state;

    public Document() {
        this.state = new Draft();
    }

    public void setState(State state) {
        this.state = state;
    }

    public void publish() {
        state.publish(this);
    }

    public void approve() {
```

You must run your code first

Applications of State Pattern

- **Workflow Systems**: It's critical to manage state transitions in business processes. The State pattern can be used to model various stages in a workflow, such as "draft," "review," "approved," and "completed." Every state may have its own rules and regulations. For example, only specific users have the ability to move a document from the "review" state to the "approved" state. The State pattern helps in the clear definition and management of these transitions, ensuring that each step adheres to business rules and processes.
- **Game Development**: A game can be in any of the following states: gameplay, paused, main menu, game-over screen, etc. The State pattern can manage these states, which also controls how the game behaves in each. For example, the pause menu is shown when the game is in the "paused" state, but game logic and animations may be put on hold. Adding new game states and controlling the transitions between them is simpler when you follow this pattern.
- **Telecommunication Systems**: Calls in telecommunication systems go through a number of states, including "dialing," "connected," "on hold," and "disconnected." The State pattern can manage these transitions effectively, ensuring that actions such as transferring or terminating a call are only available in appropriate states. For instance, if a call is not yet connected, it cannot be placed on hold.

In each of these applications, the State pattern contributes to a cleaner, more organized, and maintainable codebase by encapsulating state-specific behaviors and facilitating state transitions in a controlled manner.

Pros and Cons

Pros	Cons
Encapsulates State-Specific Behavior: Each state's behavior is in its class.	**Increased Number of Classes:** More classes to manage for each state.
Easier Maintenance: Adding new states or changing behaviors doesn't require modifying existing states.	**Context-Dependency:** States are dependent on the context, making them less reusable.
Eliminates Complex Conditional Logic: Avoids conditional complexity in the context class.	**Complexity:** Initial setup and understanding of the pattern can be complex.
Improves Readability: Clear structure for state transitions.	**Overhead:** Additional overhead in terms of memory and runtime for small use cases.

The State design pattern is an effective tool for managing state-dependent behavior in a clean and maintainable manner. It's especially helpful when an object's behavior varies greatly depending on its state. Understanding this pattern can significantly reduce the complexity of code structure in complex systems.

← Previous
Observer Pattern

Next →
Strategy Pattern

☐ Mark as Completed